



IDA DRIVING AI · 2026

Programming Languages for **AI, not humans**

What changes when the **machine** is the primary author — and what
an AI-first language has to look like for the code to be cheap to **debug**,
replay, and **fix**.

Mark Edmondson · Holosun ApS
ailang.sunholo.com · sunholo.com

You just heard how teams put AI coding into the daily workflow. Now look at what the AI is writing... in.

The **AI Coding** Revolution

90%

AI-written code predicted

Dario Amodei, Mar 2025

30 - 50%

AI-written code at Big Tech

Google & Microsoft, 2025-26

100%

Some engineers today

Boris Cherny, 2026

Dario Amodei

Confirmed 90% prediction — "I don't write any code anymore." (Mar 2026)

Boris Cherny

Ships 10-30 PRs/day, zero manual code. Created Claude Code. (Feb 2026)

Andrej Karpathy

Coined "vibe coding," now full agentic — hasn't typed code since Dec 2025.

Jensen Huang

"The programming language is human. Everybody knows human." (Jun 2025)

The question is no longer *whether* AI will write code.
It's what humans will do when it does.

What the Builders Say

Independent voices — no AI products to sell



"The python visualizer has been basically written by vibe-coding. Is this much better than I could do by hand? Sure is."

Linus Torvalds shifted
Creator of Linux & Git · Jan 2026



"You can't let the slop and cringe deny you the wonder of AI. This is the most exciting thing we've made computers do since we connected them to the internet."

DHH shifted
Creator of Ruby on Rails · Jan 2026



"90% of my skills dropped to \$0. The leverage for the remaining 10% went up 1000x. I need to recalibrate."

Kent Beck evolved
Creator of TDD & Extreme Programming · 2023-2026



"AI slop is a Denial-of-Service attack on open source maintainers." Shut down curl's bug bounty over AI-generated fake reports — but acknowledges AI finds deep bugs no tool ever could.

Daniel Stenberg nuanced
Creator of curl · FOSDEM Feb 2026



"'Coding' was never the source of value. Problem solving is the core skill. AI training on open source code magnifies the value of the gift."

John Carmack
Creator of Doom & Quake · Mar 2026

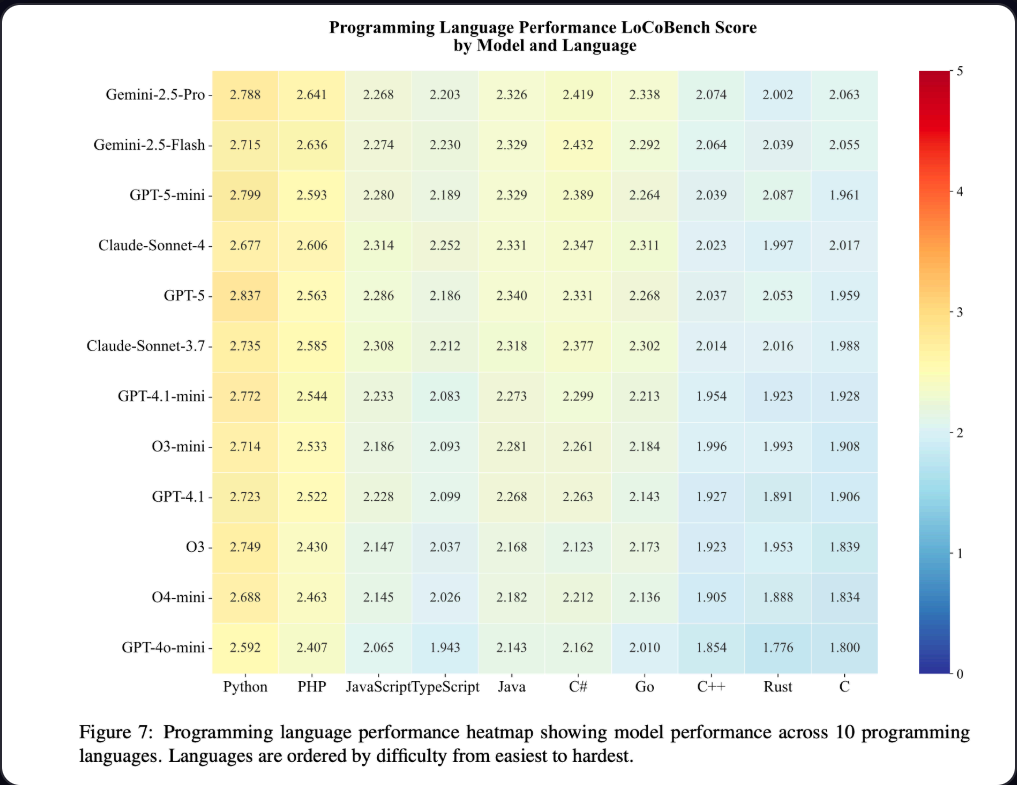


"Our job is not to type code into a computer. Our job is to deliver systems that solve problems." Coined "Deep Blue" for the existential dread developers feel.

Simon Willison
Co-creator of Django · Feb 2026

If AI Writes All the Code... Does **Language** Matter?

LoCoBench: 8,000 scenarios, 10 languages, 12 models, 10K–1M token contexts



Two forces

Training data bias favours popular languages (Python, PHP). But the consistent ordering across all 12 models reveals **systematic structural challenges** too — it's not just familiarity.

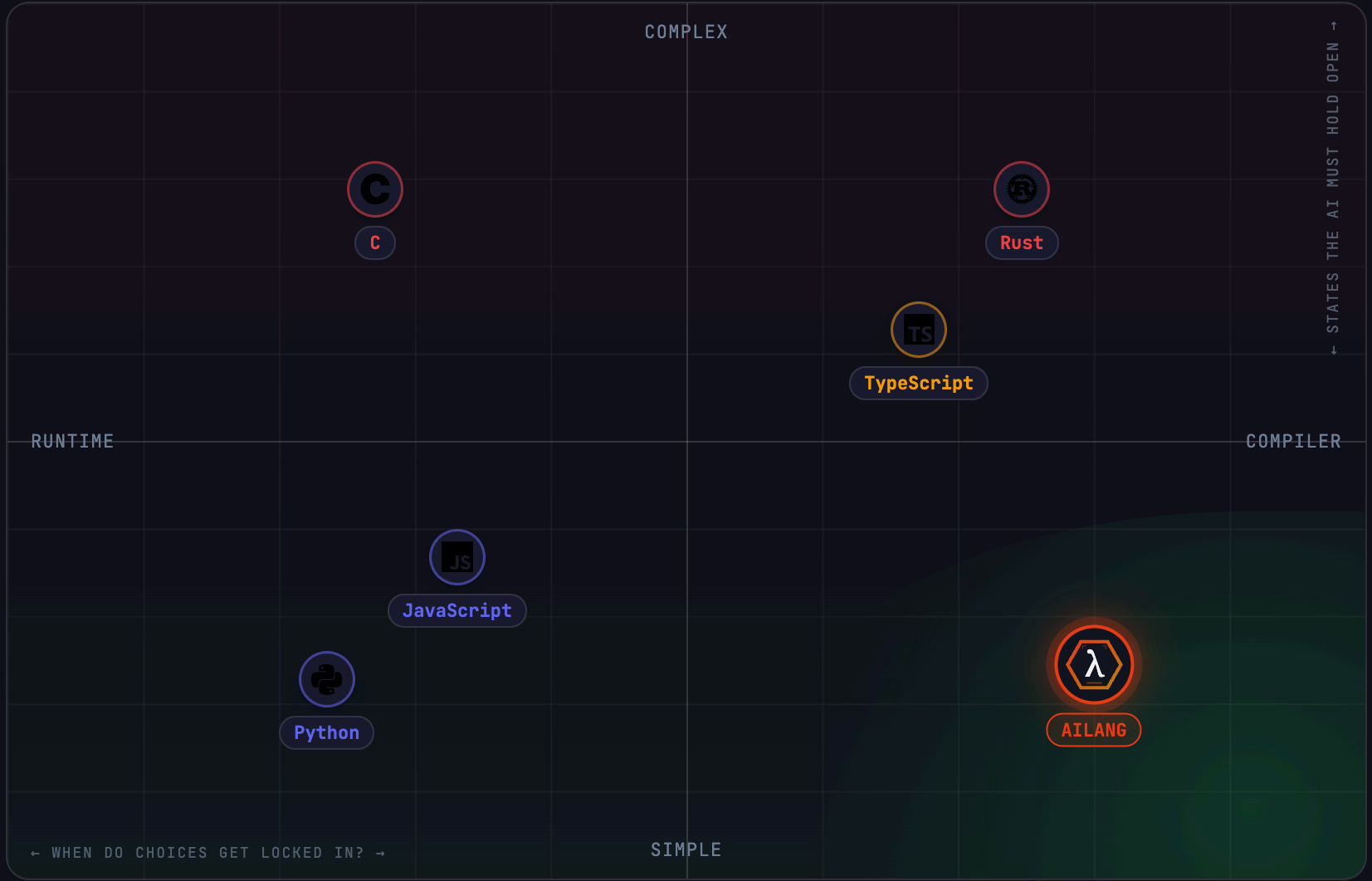
TS < JS

TypeScript scores *lower* than JavaScript despite having types. Types alone don't help if they add complexity.

C, Rust: worst

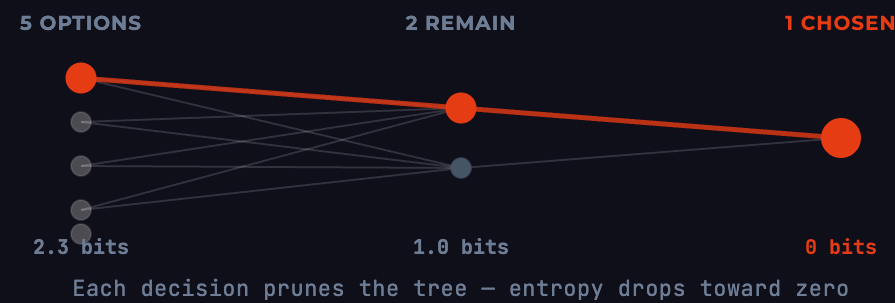
Systems languages score lowest across every model. Headers, manual memory, macros — implicit complexity that no amount of training data overcomes.

Explicitness Without Complexity



The Decision Space

Every software project starts as a vast space of **possible paths**. Each decision — a framework choice, an error strategy, an API design — eliminates alternatives and narrows what remains. **Entropy** measures how many meaningful choices are still open.



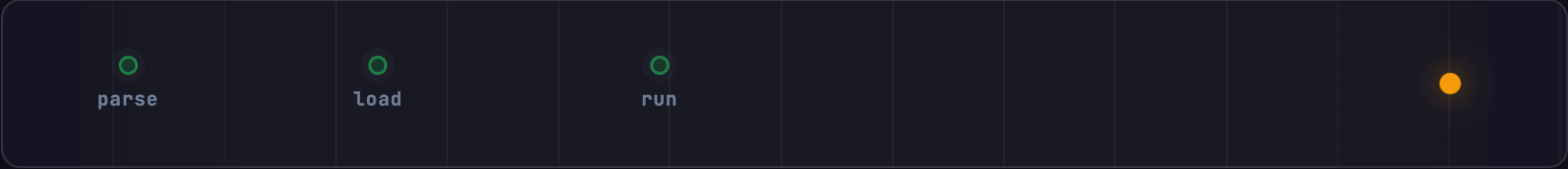
The key question isn't *how much* entropy a project has — it's *when* that entropy gets resolved. Decisions made at design time cost tokens. Decisions deferred to runtime cost debugging, deployments, and outages.

Press [→](#) to explore interactively

Where Does the Error Land?

The further the error fires, the longer the loop back. Static typing keeps that loop *tight*.

DYNAMIC
Python / JS



STATIC
AILANG



TIGHT LOOP · ~1S

compile time

runtime

AILANG catches it at the gate. Python only knows once it's inside – and the fix loop pays for it.

TIGHT FIX LOOP

AI Cannot Hallucinate a Network Call

```
export func process(path: string) → string ! {IO @limit=3, FS}
```

- ✔ IO – Can print to console
- ✔ @limit=3 – Maximum 3 times
- ✔ FS – Can read/write files
- ⊗ No Net – CANNOT touch the network
- ⊗ No DB – CANNOT access a database

```
$ ailang run --caps IO,FS process.ail
```

Only IO and FS granted at runtime. Attempting Net or DB → immediate rejection.

The compiler enforces this. Not documentation. Not discipline.

The compiler.

AI writes the code. Z3 proves it.

AILANG - CONTRACTS

```
-- ✓ This one verifies
export func calculateTax(income: int) → int ! {}
requires { income ≥ 0 }
ensures { result ≥ 0 }
{ income / 5 }

-- ✗ Subtle bug: what if price < discount?
export func applyDiscount(price: int, discount: int) → int ! {}
requires { price ≥ 0, discount ≥ 0 }
ensures { result ≥ 0 }
{ price - discount }
```

```
$ ailang verify --verbose billing.ail
```

```
✓ VERIFIED calculateTax 6ms
```

```
✗ VIOLATION applyDiscount
```

Counterexample:

```
price: Int = 0
```

```
discount: Int = 1
```

```
-- 0 - 1 = -1, violates ensures { result ≥ 0 }
```

NEURAL

LLM writes AILANG with `requires` / `ensures`



SYMBOLIC

Z3 SMT solver proves the contract for *all* inputs



FIX

Counterexample tells the AI exactly what to repair

This is neurosymbolic programming — coupled, on purpose. The neural side writes the code. The symbolic side proves it. No tests, no sampling — the solver reasons about every possible execution and hands back a concrete counterexample when it can't.

Code the AI Can Hold in One Context

No loops. Only pattern matching. Every branch is on screen — no mutable loop state to remember.

 PYTHON

```
total = 0
for item in items:
    if item.active:
        total += item.value
```

 AILANG

```
pure func total(xs: List[Item]) → int {
  match xs {
    [] ⇒ 0,
    ::(x, rest) ⇒
      if x.active then x.value + total(rest)
      else total(rest)
  }
}
```

Every branch is local. The model doesn't have to track an accumulator across loop iterations or wonder if it terminates. Compiler-checked exhaustiveness means fewer tokens spent on reasoning about state — and a one-turn fix when a branch is missing.

What's in Scope Is What's on Screen

The same line of code — two very different mental loads on the AI.

ONE LINE. WHAT'S IN SCOPE?

 PYTHON

```
import requests
```

WHAT THE AI MUST REASON ABOUT

requests.get

requests.post

urllib3.* ?

charset_normalizer ?

idna ?

certifi.* ?

ssl ?

socket ?

http.client ?

json ?

os.environ ?

logging ?

warnings ?

io.BytesIO ?

... and more, transitively

opaque scope

2 named · 12+ reachable through the package

The model spends tokens **guessing what else is in scope**. Sometimes it hallucinates symbols that don't exist; sometimes it uses ones that do but it shouldn't.

ONE LINE. WHAT'S IN SCOPE?

 AILANG

```
import std/net (get, post)
```

WHAT THE AI MUST REASON ABOUT

get

post

full inventory

2 named · 2 reachable

The file **is** the inventory. Nothing arrives transitively. Whatever the AI cannot name, it cannot call.

Complexity Is in the Eye of the Beholder

Every AILANG choice has two readers. Same code — two very different reactions.

THE DESIGN CHOICE	HUMAN <i>grumbles</i>	AI AGENT <i>thanks you</i>
Static, fully-inferred types	"Why annotate?"	> Pre-runtime, parseable errors
No loops, only recursion	"Verbose"	> Total, analysable iteration
Explicit effects in types	"Noisy signatures"	> Can't hallucinate capabilities
No transitive imports	"Tedious"	> Bounded context window
No mutable state	"Inconvenient"	> No temporal reasoning needed
One canonical syntax	"Less expressive"	> Zero ambiguity
Structured errors	"More boilerplate"	> Parseable feedback for self-repair
👤 7 COMPLAINTS · SAME LANGUAGE. · 🤖 7 WINS		

The Development Loop

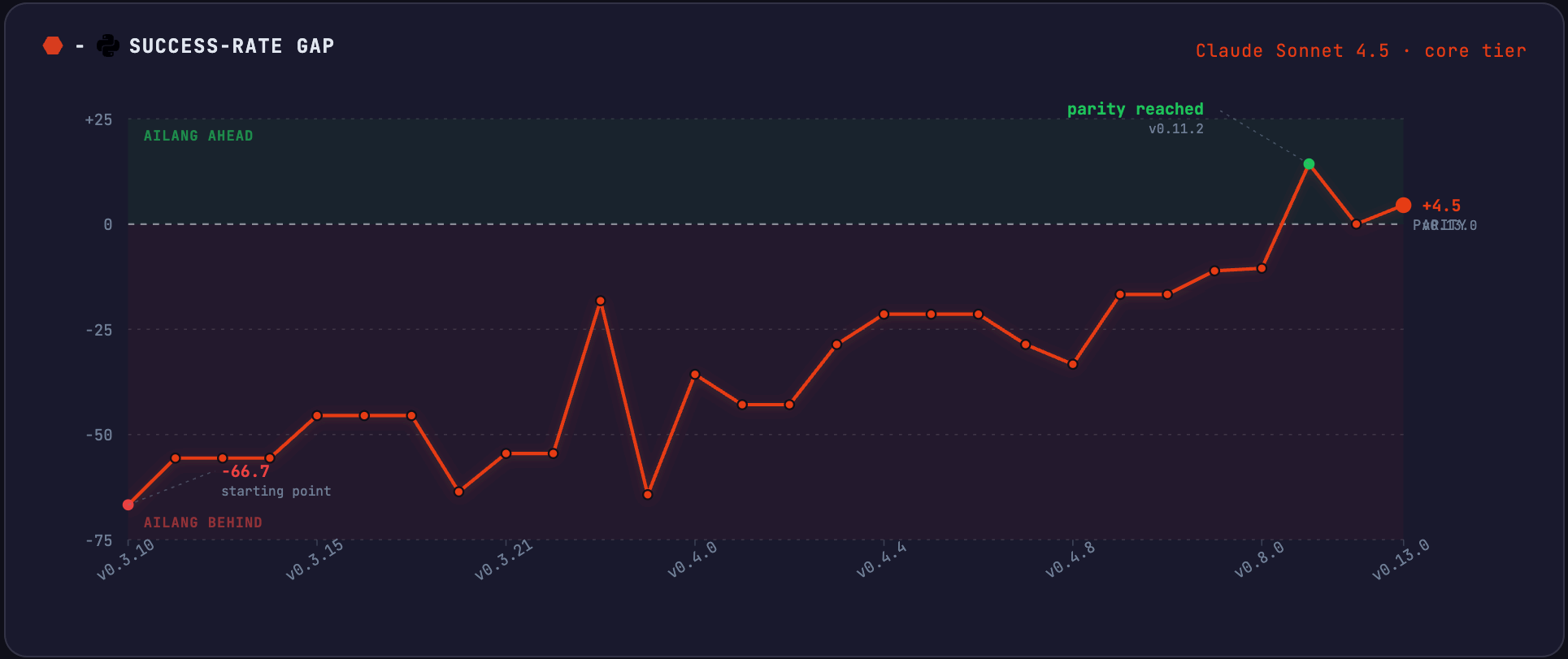
SIX STAGES · THE BRIEF LOOPS BACK · NOTHING RUNS WITHOUT IT



A human writes the brief. Nothing else runs until it's approved.

Does it **work**?

AILANG started **65 points behind** Python on Claude Sonnet 4.5. Watch the gap close.



data: ailang.sunholo.com/benchmarks/latest.json • snapshot 2026-05-11

From **-65 to parity** on Claude Sonnet 4.5. The dips near the top aren't regressions — that's where saturated benchmarks get retired and harder ones come in. The bar keeps rising even after we catch up.

🚀 AILANG IN PRODUCTION

100% AI-coded apps. Running today.

Real WASM modules in your browser — no Python, no server — every extraction validated by a contract the compiler refuses to ship without.



DOCUMENT INTELLIGENCE

AILANG Parse

DOCX, PPTX, XLSX, PDF & images parsed **directly in your browser**. Track changes, merged cells, comments — preserved, not flattened.

69 files 15 formats
3 / 3 track changes (vs 0 / 5 others)

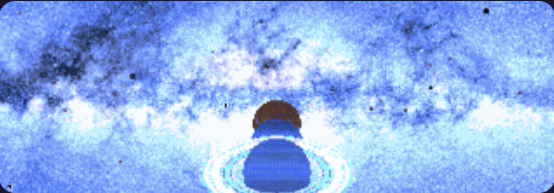


STREAMING & VOICE

Voice DocParse + Website Builder

Talk to a document via Gemini Live. Or describe a site and watch it build — AILANG WASM modules driving the whole pipeline.

WASM runtime Gemini Live bidi audio
no server



GAME ENGINE

Stapledon's Voyage

A hard-sci-fi relativity simulator. AILANG drives the **game logic** — special & general relativity, flybys, time dilation. Not a documents demo.

AILANG game core SR + GR physics
open source

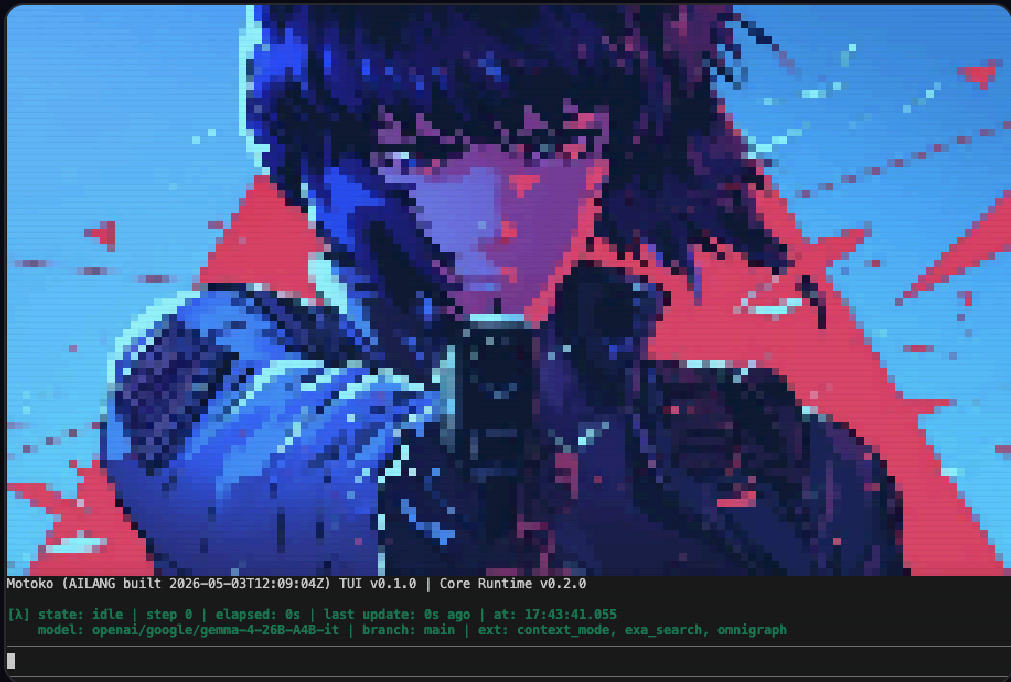
Everything you see is

100% AI-coded in AILANG

— compiler, type checker, effects, Z3 verification, WASM runtime, and stdlib included.

sunholo.com/ailang-demos

A coding harness that **grows itself** — from a verifiable core.



built by [@arniwesth](#)

λ_ motoko

Phoenix architecture

Start with a small, **verifiable** AILANG core. The agent uses the core to write more AILANG — tools, extensions, even *parts of itself*. Every step goes through the compiler. If it can't be verified, it doesn't ship.



Verifiable core

AILANG types · contracts · effects all checkable



Self-verifying loop

the agent runs its own compiler · fixes its own errors



Self-extending

AI builds new tools in AILANG · verified before they're trusted

SELF-CORRECTING LOOP

When the AI gets it **wrong**... the language tells it.

Live session · **openrouter/z-ai/glm-5** · task: wire an MCP subprocess into AILANG ·
2026-05-07

~/motoko_agent · session_2026-05-07.jsonl

step 61

AGENT

Perfect! I have successfully implemented the task.  Implementa

no human in the loop · 8 steps of self-repair after the parse error



Don't ask AI to learn your language. **Build a language AI can speak.**

Cheaper to debug. Replayable. Verifiable.



STATIC TYPES • THE AI SAFETY NET

Pre-runtime, parseable errors mean one-turn fixes — not production crashes.



NEUROSymbOLIC VERIFICATION

AI writes the code, Z3 proves the contracts. Counterexamples beat tests.



WORKS IN ANY STACK

Tight loops, explicit context, structured errors — push for these where you are.

 ailang.sunholo.com language • playground • demos

 mcp.ailang.sunholo.com live MCP for Claude / Cursor / Cline

 github.com/sunholo-data/ailang source • compiler • stdlib

 github.com/arniwesth/motoko_agent the motoko coding harness

👋 THANK YOU

Let's keep **talking**



Mark Edmondson

FORWARD DEPLOYED ENGINEER · SUNHOLO · COPENHAGEN



LINKEDIN
markpeteredmondson



WEBSITE
sunholo.com



EMAIL
m@sunholo.com



AILANG
ailang.sunholo.com



WORKSHOPS
sunholo.com/workshops



SUBSTACK
markedmondson

Questions, ideas, or want to keep the conversation going? *Any of the above works.*